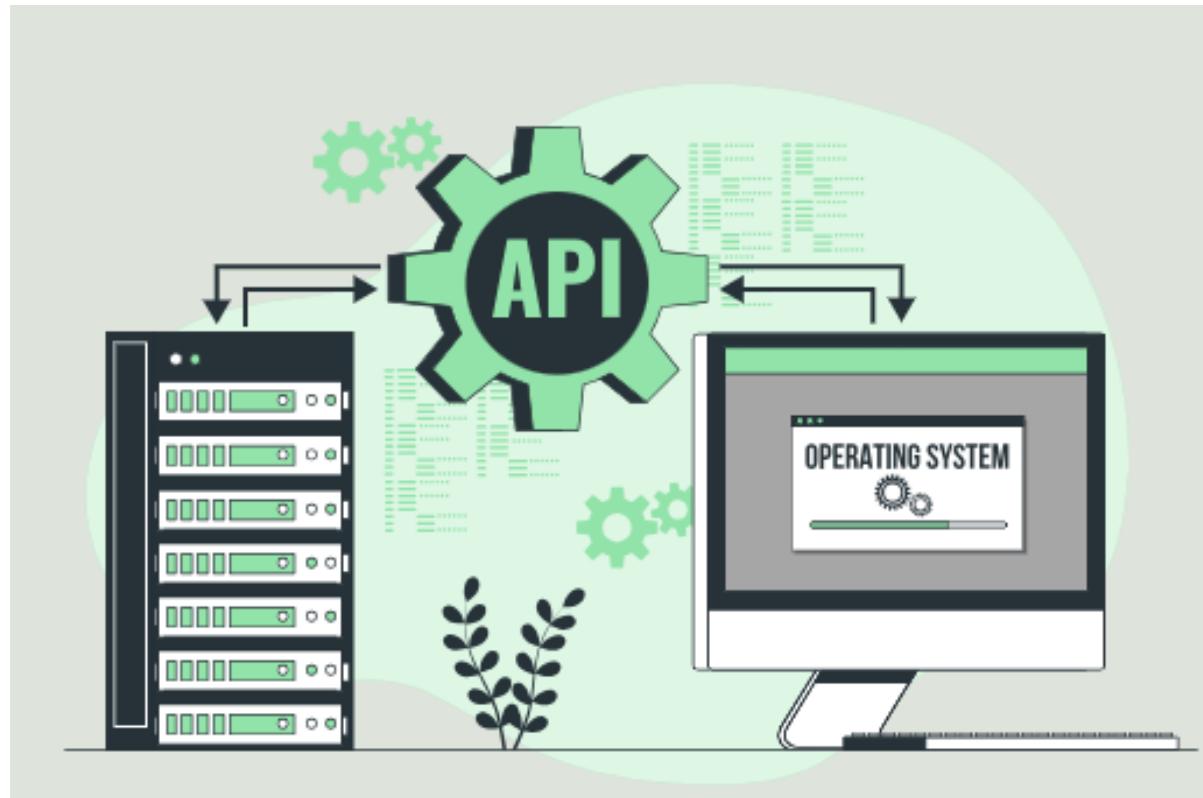


Тестирование API



Тестирование API — это тип тестирования ПО, который анализирует программный интерфейс приложения (API) для проверки того, что он соответствует ожидаемой функциональности, безопасности, производительности и надежности.

Тестирование API фокусируется на анализе бизнес-логики приложения, а также на данных и на безопасности.

В Agile-средах модульные тесты и тесты API предпочтительнее, чем тесты графического пользовательского интерфейса (GUI), поскольку их легко поддерживать, и они более эффективны.

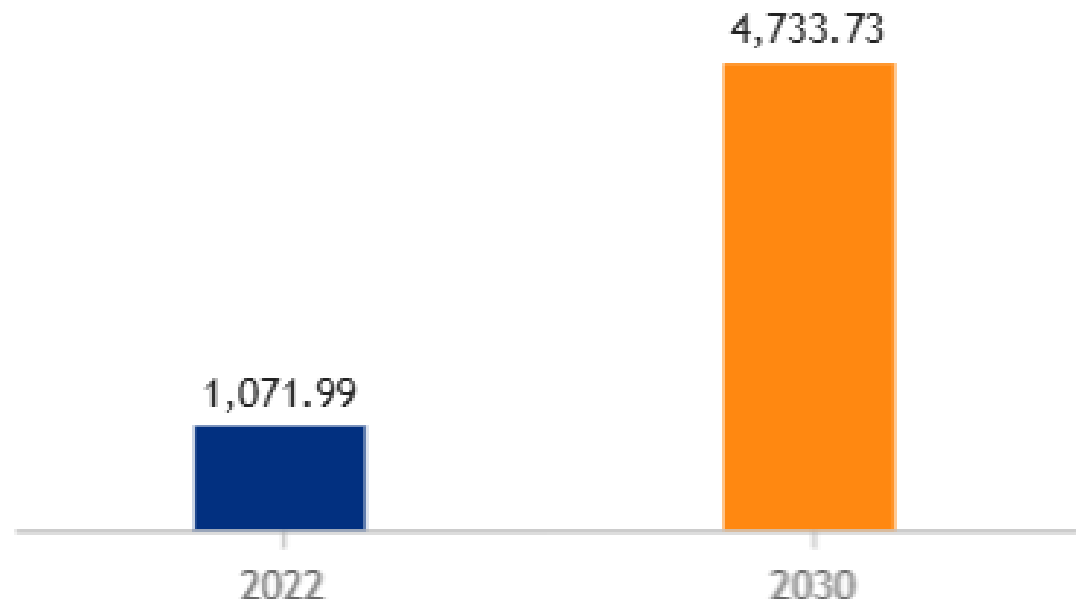
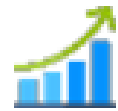
НО, такое тестирование может быть сложным для небольших QA-команд, нанимаемых «под проект».

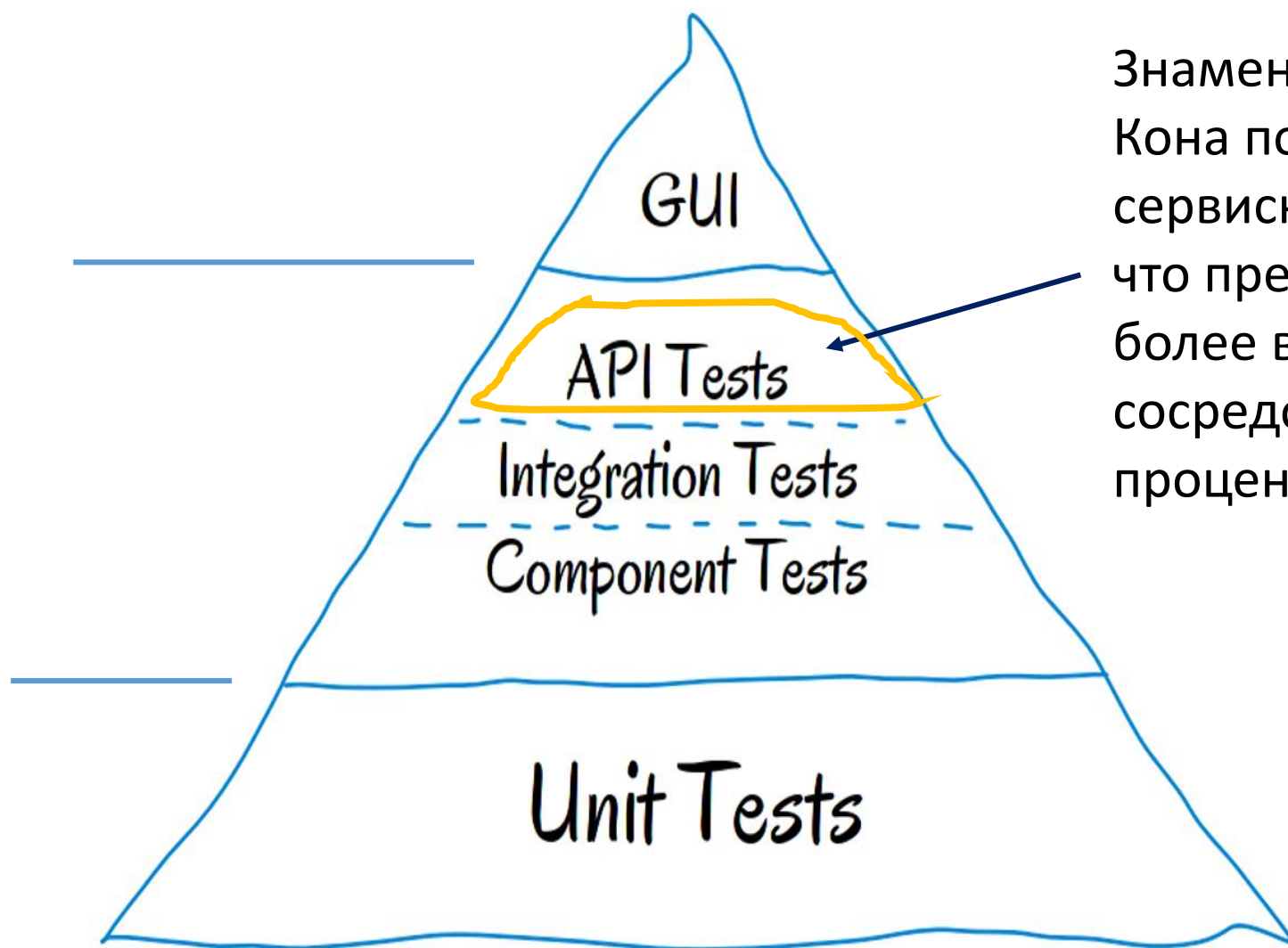
Все больше компаний переходят на использование микросервисов, а большинство микросервисов используют API, поэтому, рынок тестирования API предсказуемо будет расти.

Глобальный рынок тестирования API

Размер рынка в млрд долларов США

Среднегодовой темп роста: 20.40%





Знаменитая пирамида тестов Майка Кона помещает тесты API на сервисный уровень (интеграционный), что предполагает, что около 20% или более всех тестов должны быть сосредоточены на уровне API (точный процент зависит от потребностей).

Проектирование тестирования API

Вопросы, которые следует рассмотреть на этом этапе:

- ✓ Каковы функциональные возможности API?
- ✓ Какие конечные точки доступны для тестирования?
- ✓ Какие коды ответов ожидаются для успешных запросов
- ✓ Какие коды ответов ожидаются в случае неудачных запросов?
- ✓ Какое сообщение об ошибке ожидается в теле неудачного запроса?
- ✓ Какие инструменты тестирования API следует использовать?

Этапы тестирования API

Для каждого запроса API тест должен выполнить следующие действия:

1. Проверка корректности кода состояния HTTP. Например, создание ресурса должно возвращать 201 CREATED, а запрещенные запросы должны возвращать 403 FORBIDDEN и т. д.
2. Проверка полезной нагрузки ответа. Проверьте правильность тела JSON, имен, типов и значений полей ответа, в том числе в ответах на ошибочные запросы.
3. Проверка заголовков ответа. Заголовки HTTP-сервера влияют как на безопасность, так и на производительность.
4. Проверка правильности состояния приложения. Это необязательно и применяется в основном к ручному тестированию или когда пользовательский интерфейс или другой интерфейс можно легко проверить.
5. Проверка времени ответа. Если операция была завершена успешно, но заняла неоправданно много времени, тест не пройден.

Пример: тестирование RESTful API для системы управления задачами

В общих чертах тестирование API содержит следующие шаги

Предположим, что этот API предоставляет следующие методы:

- **GET /tasks** — получить список всех задач;
- **GET /tasks/{id}** — получить информацию о конкретной задаче;
- **POST /tasks** — создать новую задачу;
- **PUT /tasks/{id}** — обновить информацию о задаче;
- **DELETE /tasks/{id}** — удалить задачу.

1

Определить требования

Изучить документацию API, чтобы понять, какие поля должны быть в задаче, как она должна быть создана и обновлена, ожидаемые коды состояния и структуру ответов.

2

Создать тестовые случаи, например:

- Тест случая GET /tasks — отправить GET-запрос на /tasks и убедиться, что полученный ответ содержит список задач.
- Тест случая POST /tasks — отправить POST-запрос на /tasks с тестовыми данными для создания новой задачи и убедиться, что задача успешно создана и возвращается правильный код состояния (например, 201 Created).
- Тест случая PUT /tasks/{id} — создать тестовую задачу, получить её идентификатор, затем отправить PUT-запрос на /tasks/{id} с обновлёнными данными и убедиться, что задача обновлена и возвращается правильный код состояния (например, 200 OK).

3

Настроить окружение

Установить Postman для отправки запросов и проверки ответов API.

Шаги тестирования
могут быть такими:

4

Отправить запросы

С использованием Postman отправить запросы из тестовых случаев к API.

5

Проверить ответы

Проверить ответы от API, используя ожидаемые результаты, указанные в тестовых случаях. Например, убедиться, что **возвращается правильный код состояния, структура ответа соответствует ожидаемой и значения полей задачи корректны.**

6

Обработать ошибки

Проверить, как API обрабатывает ошибки, отправляя некорректные запросы. Убедиться, что **возвращаются соответствующие коды и описания ошибок.**

7

Сгенерировать отчёты

Создавать отчёт о результатах после каждого выполнения тестов. Включить в него информацию о прошедших и не прошедших тестах, кодах состояния, ответах, ошибках, и другие полезные данные.

8

Регулярно проводить повторное тестирование

Особенно после внесения изменений в код API или его окружения, для обнаружения проблем и подтверждения работоспособности API.

Пример:

Предположим, что подмножеством нашего API является конечная точка **/users**, которая включает следующие вызовы API:

Вызов API	Действие
GET /users	Просмотреть всех пользователей
GET /users?name={username}	Получить пользователя по имени пользователя
GET /users/{id}	Получить пользователя по ID
GET /users/{id}/configurations	Получите все конфигурации для пользователя
POST /users/{id}/configurations	Создать новую конфигурацию для пользователя
DELETE /users/{id}/configurations/{id}	Удалить конфигурацию для пользователя
PATCH /users/{id}/configuration/{id}	Обновить конфигурацию для пользователя

Проверить все конечные точки GET, которые позволяют фильтровать, сортировать, исключать и ограничивать дополнительные параметры запроса для фильтрации, сортировки и разбивки на страницы тела ответа.

Позитивные тесты

Проверить код состояния:	1. Все запросы должны возвращать код состояния HTTP 2XX. 2. Код статуса возвращен согласно спецификации: – 200 OK для запросов GET – 201 для запросов POST или PUT, создающих новый ресурс – 200, 202, или 204 для операции DELETE и т. д.
Подтвердить загрузку ответа:	1. Ответ - это правильно сформированный объект JSON. 2. Структура ответа соответствует модели данных (проверка схемы: имена и типы полей соответствуют ожидаемым, включая вложенные объекты; значения полей соответствуют ожидаемым; поля, не допускающие значения NULL, не являются пустыми и т. Д.)
Проверить состояние:	1. Для запросов GET убедитесь, что система НЕ ИЗМЕНИЛА СОСТОЯНИЕ (идемпотентность) 2. Для операций POST, DELETE, PATCH, PUT: - Убедитесь, что действие в системе было выполнено правильно: * Выполнение соответствующего запроса GET и проверка ответа * Обновите пользовательский интерфейс в веб-приложении и проверьте новое состояние (применимо только для ручного тестирования)
Проверить заголовки:	1. Убедитесь, что заголовки HTTP соответствуют ожидаемым, включая: content-type, connection, cache-control, expires, access-control-allow-origin, keep-alive, HSTS, и другие стандартные поля заголовка - согласно спецификации 2. Убедитесь, что информация НЕ утекает через заголовки (например X-Powered-By заголовков не отправляется пользователю).
Проверка производительности:	Ответ получен своевременно (в разумные ожидаемые сроки) - как определено в плане тестирования.

Позитивные тесты + дополнительные параметры эндпоинта: сортировка, фильтрация, ограничения...

Проверить код состояния:	Как в пункте #1
Проверить правильность загрузки:	1. Проверьте структуру и содержание ответа, как в пункте #1. 2. Кроме того, проверьте следующие параметры: – фильтрация: убедитесь, что ответ фильтруется по указанному значению. – сортировка: укажите поле для сортировки, проверки опций по возрастанию и убыванию. Убедитесь, что ответ отсортирован в соответствии с выбранным полем и направлением сортировки. – пропуск: убедитесь, что указанное количество результатов с начала выборки данных пропущено – лимитирование: убедитесь, что размер набора данных ограничен указанным пределом. – лимитирование + пропуск: тестовая пагинация. 3. Проверьте комбинации всех необязательных полей (поля + сортировка + ограничение + пропуск) и проверьте ожидаемый ответ.
Проверка состояния:	Как в пункте #1
Проверка заголовков:	Как в пункте #1
Проверка производительности:	Как в пункте #1

Негативные тесты

Проверить коды состояния:	1. Убедитесь, что отправлен ошибочный код состояния HTTP (НЕ 2XX) 2. Убедитесь, что код состояния HTTP соответствует типу ошибки, как определено в спецификации.
Проверить правильность загрузки:	1. Убедитесь, что получен ответ об ошибке 2. <u>Убедитесь, что формат ошибки соответствует описанному в спецификации.</u> например, ошибка - это допустимый объект JSON или пустая строка (как определено в спецификации) 3. Убедитесь, что есть четкое и понятное поле сообщения об ошибке/описание ошибки. 4. Убедитесь, что описание ошибки правильное для данного типа ошибки и соответствует спецификации
Проверка заголовков:	Как в пункте #1
Проверка производительности:	Убедитесь, что ошибка получена своевременно (в разумные ожидаемые сроки)

Негативное тестирование – валидный ввод данных

Выполнять вызовы API с допустимыми входными данными, которые пытаются выполнить незаконные операции. то есть:

- Попытка создать ресурс с уже существующим именем (например, пользовательская конфигурация с тем же именем)
 - Попытка удалить ресурс, который не существует (например, конфигурация пользователя без такого идентификатора)
 - Попытка обновить ресурс недопустимыми действительными данными (например, переименовать конфигурацию в существующее имя)
 - Попытка незаконной операции (например, удаление конфигурации пользователя без разрешения).
- И так далее.

Негативное тестирование - неверные входные данные

Требуется выполнять вызовы API с недопустимым вводом, например:

- Токен авторизации отсутствует или недействителен.
 - Отсутствуют необходимые параметры
 - Недействительное значение для параметров конечной точки, например:
 - * Неверный UUID в параметрах пути или запроса
 - * загрузка с недопустимой моделью (нарушает схему)
 - * загрузка с неполной моделью (отсутствующие поля или обязательные вложенные сущности)
 - * Недействительные значения во вложенных полях сущности
 - * Неподдерживаемые методы для конечных точек.
- И так далее.

Деструктивное тестирование

Умышленно попытаться вывести API из строя, чтобы проверить его надежность:

См. след. слайд

Деструктивное тестирование

Умышленно попытаться вывести API из строя, чтобы проверить его надежность:

- Некорректный контент в запросе
- Неверный тип содержимого в загрузке
- Контент с неправильной структурой
- Переполнение значений параметров. Например.:
 - а) Попытка создать пользовательскую конфигурацию с заголовком более 200 символов
 - б) Попытка GET пользователя с недопустимым UUID длиной 1000 символов
 - в) Переполнение тела ответа
 - г) огромный JSON в теле запроса
- Проверка граничных значений
- Пустые полезные данные
- Пустые подобъекты в загрузочных данных
- Недопустимые символы в параметрах или загрузочных данных

- Использование неправильных заголовков HTTP (например, Content-Type)

- Простые тесты параллелизма - одновременные вызовы API, которые пишут в одни и те же ресурсы (DELETE + PATCH, и т.п.)

Другое исследовательское тестирование

Типы тестирования API

1. Тестирование методов

Каждый метод API проверяют по отдельности — чтобы убедиться, что они работают правильно и возвращают ожидаемые результаты. Тестирование включает проверку входных данных, выполнение операций и проверку вывода.

2. Тестирование взаимодействий

Проверка взаимодействия API с другими API, компонентами программы и сервисами. Показывает, что он успешно отправляет и получает данные, а также обрабатывает различные сценарии использования.

3. Тестирование авторизации и аутентификации

Проверка доступа к API: как работают механизмы авторизации, кто и к каким функциям и данным имеет доступ.

4. Тестирование обработки ошибок

Проверка поведения API в случае непредвиденных ситуаций и ошибок — например, передачи некорректных данных. Позволяет убедиться, что API правильно обрабатывает исключения и передаёт в программу верные коды ошибок.

5. Тестирование производительности

Проверка работы API при повышенных нагрузках — его пропускной способности и производительности. Позволяет убедиться, что API не «отвалится» в случае повышенного спроса на программу.

6. Тестирование безопасности

Проверка уязвимостей безопасности API, которая помогает предотвратить утечку данных или несанкционированный доступ. Проверяются меры безопасности API и проводятся тесты на проникновение для установления возможных уязвимостей.

Принципы тестирования API

✓ Использовать правильные и разнообразные входные данные

При тестировании важно использовать разные типы данных, граничные значения, некорректные данные. Это помогает убедиться, что API правильно обрабатывает все возможные входные сценарии.

✓ Автоматизировать тестирование

В тестировании API многое можно автоматизировать: например, проверку отдельных функций или обработку ошибок.

✓ Проводить непрерывные тесты

Если в компании есть процессы CI/CD, важно включить в них тестирование API. Это позволит регулярно проверять его работоспособность и получать обратную связь о проблемах сразу после их возникновения.

✓ Проверять безопасность

Поскольку API часто поставляются со стороны, важно тщательно тестировать их на уязвимости и проверять механизмы аутентификации, чтобы и API, и основная система были защищены от потенциальных угроз и атак.

Преимущества тестирования API

- **Для автоматизации тестирования API требуется меньше кода**, чем для автоматизированных тестов GUI, что обеспечивает более быстрое тестирование и более низкую общую стоимость.
- Тестирование API позволяет разработчикам получать доступ к приложению без UI, помогая им **выявлять ошибки на ранних этапах жизненного цикла разработки**. Это экономит деньги.
- **Тесты API не зависят от технологии и языка**. Обмен данными осуществляется с использованием JSON или XML, и содержит HTTP-запросы и ответы.
- Тесты API используют экстремальные условия и входные данные при анализе приложений. Это **помогает устранить уязвимости и защищает приложение от вредоносного кода и поломок**.
- Тесты API могут быть интегрированы с тестами GUI. Например, интеграция может позволить создавать новых пользователей в приложении до выполнения теста GUI.
- **Тестирование API можно эффективно автоматизировать**, что сокращает ручные усилия во время регрессионного тестирования и существенно экономит время.
- **Автоматизированное тестирование API не зависит от конкретного типа языка программирования**. Оно позволяет инженерам по обеспечению качества использовать любой язык программирования, поддерживающий технологии XML и JSON, независимо от языков приложений.

Проблемы тестирования API

- ✓ **Выбор параметров** требует проверки параметров, отправляемых через запросы API, — процесс, который может быть сложным. Однако необходимо гарантировать, что все данные параметров соответствуют критериям проверки, таким как использование соответствующих строковых или числовых данных, назначенный диапазон значений и соответствие ограничениям длины.
- ✓ **Комбинирование параметров** может оказаться сложной задачей, поскольку каждую комбинацию необходимо протестировать на предмет наличия проблем, связанных с конкретными конфигурациями.
- ✓ **Последовательность вызовов.** Чтобы гарантировать правильную работу системы, каждый вызов должен появляться в определенном порядке. Это может быть сложным, особенно при работе с многопоточными приложениями.
- ✓ **Тестировщики должны уметь писать код.**

Распространенные баги

- Баги в бизнес-логике, например финансовых платежей в банковском приложении
- Баги хранения данных и их передачи, например неспособность сохранять файлы с кириллическими символами в именах
- Нестабильное поведение, например API недоступен время от времени, или увеличивается время ответа до неприемлемого
- Неправильная обработка исключений, например API не возвращает ожидаемый код ошибки, если что-то нужное не найдено
- Неспособность отрабатывать негативные сценарии

1. Устаревший API Stripe

Топ-5 инцидентов,
связанных с
API, в 2025 году

<https://equixly.com/blog/2025/09/08/2025-top-5-api-incidents/>

Злоумышленники обнаружили устаревшую конечную точку (`/v1/sources`), которая по-прежнему была подключена к внутренним системам проверки платежей. Эта конечная точка не имела средств защиты, присущих современным API Stripe, таких как расширенные ограничения скорости запросов и алгоритмы обнаружения мошенничества.

В этом нет ничего необычного. В среднем организации проверяют на уязвимости лишь 38% своих API, оставляя устаревшие или теневые API в опасном положении.

Поскольку этот API-интерфейс не входил в официальный реестр, Stripe не подвергал его такому же тщательному тестированию и мониторингу, как другие API. Это позволило злоумышленникам совершать часто используемые, но малоэффективные запросы в течение длительного времени оставаться незамеченными стандартными системами оповещения о безопасности.

2. McDonald's / Paradox.ai

Топ-5 инцидентов,
связанных с
API, в 2025 году

<https://equixly.com/blog/2025/09/08/2025-top-5-api-incidents/>

Инцидент с данными платформы McHire компании McDonald's стал классическим примером риска для безопасности, связанного с третьими сторонами. Уязвимости в системе Paradox.ai — компании, предоставляющей чат-бота на основе искусственного интеллекта, используемого для проверки кандидатов на работу, — привели к утечке конфиденциальных данных, принадлежащих крупной корпорации.

Специалисты по безопасности, изучавшие платформу McHire, обнаружили две критические уязвимости:

- **Административная учетная запись на платформе McHire была защищена стандартной комбинацией имени пользователя и пароля: 123456:123456** . Слабые учетные данные предоставили им административный доступ к тестовой учетной записи ресторана. Эта учетная запись, которую следовало вывести из эксплуатации за несколько лет до этого, не поддерживала многофакторную аутентификацию и обеспечила первоначальный доступ к системе.
- Оказавшись внутри, исследователи обнаружили серьезную ошибку авторизации на уровне объектов в API, управляющем данными заявителей. **Конечная точка API для получения журналов чата и личной информации заявителя использовала в запросе последовательный идентификационный номер**. Увеличивая этот номер, исследователи могли перебирать записи заявителей без каких-либо проверок авторизации.

В результате этого инцидента была раскрыта личная информация примерно **64 миллионов заявителей** .

Инструменты тестирования API



- ✓ **Postman.** Позволяет создавать, отправлять и тестировать HTTP-запросы и получать ответы от API. Предоставляет возможность создания автоматизированных тестов, генерации документации и совместной работы в команде.



- ✓ **SoapUI.** Позволяет тестировать и отлаживать SOAP и REST API: создавать и отправлять запросы, автоматизировать тестирование, генерировать тестовые отчеты, мониторить производительность.



- ✓ **JMeter.** Позволяет отправлять HTTP-запросы и проводить нагрузочное тестирование, чтобы проверить, как API справляется с высокими нагрузками, насколько оно производительно и масштабируемо.



- ✓ **REST-assured.** Java-библиотека, которая предоставляет простой и удобный способ для тестирования REST API с использованием DSL-синтаксиса. Она позволяет выполнять запросы, проверять ответы и создавать автоматизированные тесты.



- ✓ **Swagger.** Позволяет автоматически генерировать код для тестирования API и создавать автотесты.

Postman — это сервис для создания, тестирования, документирования, публикации и обслуживания API.

Он позволяет создавать коллекции запросов к любому API, применять к ним разные окружения, настраивать мок-серверы, писать автотесты на JavaScript, анализировать и визуализировать результаты запросов.

Программа поддерживает разные виды архитектуры API: HTTP, REST, SOAP, GraphQL и WebSockets. Postman вовсю используют в Twitter, WhatsApp и Imgur, но благодаря удобному графическому интерфейсу разобраться в платформе может даже новичок.

Скачать Postman можно бесплатно на официальном сайте. Есть версии под Linux, Windows и macOS.



https://reqres.in/api/users



Save



Share

POST



https://reqres.in/api/users

Send



Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

Pre-request

Post-response



```
1 pm.test("Test Add_new_user", function () {  
2   pm.response.to.have.status(201);  
3 });
```



[Learn more.](#)

Snippets

Get an environment variable

Get a global variable

Get a variable

Get a collection variable

Body

Cookies

Headers (15)

Test Results (1/1)



201 Created

399 ms

1.06 KB



e.g.



Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#)



Run Collection



All

Passed

Skipped

Failed



PASS

Test Add_new_user